

# **FORMAN CHRISTIAN COLLEGE (A CHARTERED UNIVERSITY)**



**COMP 451 B**

**SP24**

**Assignment 1**

**Daim Bin Khalid - 251686775**

**24<sup>th</sup> March , 2024**

# **INTRODUCTION**

This report delves into the implementation of a C program designed to manipulate C text files, emulating the working of compiler preprocessor. The program's objectives encompass various aspects of C programming, including file handling, character manipulation, command-line argument parsing, and compiler preprocessing.

## **Objectives:**

- Utilize command-line argument parsing to customize program behavior dynamically.
- Implement file handling operations for reading and writing text files.
- Create stages of preprocessor file cleanup consisting of removing blank lines, removing comments, and finally expanding macros.
- Harness the power of the compiler preprocessor to enhance program functionality.

The program reads a C file and performs operations such as adding line numbers, removing blank lines, removing comments, and expanding macros. It generates separate text files for each stage of preprocessor functionality. Command-line arguments play a crucial role in specifying input and output file paths, while the compiler preprocessor logic written in the program handles various stages of code cleanup with each specific function.

## **Compiler Preprocessor:**

The compiler preprocessor is an essential component of the C compilation process, responsible for performing text manipulations on the source code before actual compilation. It executes directives prefixed by the '#' symbol, transforming the source code based on specified instructions.

Key features of the compiler preprocessor include:

- Removing blank lines from text file.
- Removing comments as they are meant for users and program itself has no use for them.
- Macro expansion: Substituting macro identifiers with their corresponding definitions throughout the code, enhancing code readability and maintainability.
- Symbol definition: Defining symbolic constants using `#define`, improving code clarity and reducing magic numbers.

By leveraging the capabilities of the compiler preprocessor, developers can streamline code development, enhance code readability, and optimize program performance.

Standard C library functions serve as the backbone of this program, facilitating various tasks such as file handling, input validation, and character manipulation. These functions streamline development, ensuring efficient and reliable program execution. Below are the details of these library functions and their significance within the context of the program.

1. **<stdlib.h>**: This library provides functions for memory allocation, process control, and other utility functions. In our program, it's utilized for process control.
  - `exit()`: This function terminates the program immediately, closing all open files and releasing allocated memory. It's invoked in error handling routines to gracefully exit the program in case of invalid inputs or file operations.
2. **<stdio.h>**: The standard input-output library in C provides functions for file handling and console input and output operations. In our program, it's extensively used for file handling and console output.
  - `fopen()`: Opens a file specified by its filename and mode. It's utilized to open the input file in read mode for reading its content, as well as for error checking to ensure the file exists and can be accessed.
  - `fgetc()`: Reads a character from the specified file. It's used to read characters from the input file one by one.
  - `fgets()`: Reads a file line by line.
  - `fputc()`: Writes a single character to file.
  - `fprintf()`: Writes formatted output to a file. It's employed to write the line numbers and modified content to the output file.
  - `feof()`: Checks for the end-of-file indicator. It's utilized to determine if the end of the input file has been reached during reading.
  - `printf()`: Used to format and print output to the console. It's employed for displaying informative messages, usage instructions, and file content during program execution.
  - `fclose()`: Closes the file associated with the given file pointer. This function is invoked to release system resources after reading the file content or performing decoding operations.
3. "helper.h": Custom defined library to prevent using <string.h> library by manually handling the needed functionalities.
  - `copyString()`: Copies a string from a source pointer to a destination pointer.
  - `stringLength()`: Calculates the total length of a string.
  - `stringScanf()`: Checks if a string contains a particular substring passed to it.

# ALGORITHM & LOGIC

## Section 1: Removing Blank Lines

The process of removing blank lines from a C text file involves iterating through each line of the input file, identifying empty lines, and writing non-empty lines to the output file. This section outlines the algorithm and logic behind the `removeBlankLines` function.

### **Functionality:**

This function aims to eliminate blank lines from the content of the input file, ensuring that only lines containing text are preserved in the output file.

### **Arguments:**

- `inputFile`: Path to the input file containing text with potentially blank lines.
- `outputFile`: Path to the output file where the content without blank lines will be written.

### **Variables:**

- `readFile`: File pointer for the input file.
- `blankFile`: File pointer for the output file without blank lines.
- `buffer`: Character array to store each line read from the input file.

### **Code Snippet:**

```
void removeBlankLines(char *inputFile, char *outputFile) {
```

```
    FILE *readFile = fopen(inputFile, "r");
```

```
    if (readFile == NULL) {
```

```
        printf("Couldn't open %s\n", inputFile);
```

```
        exit(1);
```

```
    }
```

```
    FILE *blankFile = fopen(outputFile, "w");
```

```
    if (blankFile == NULL) {
```

```
        printf("Couldn't open %s\n", outputFile);
```

```
        exit(1);
```

```

}

char buffer[256];
while (fgets(buffer, sizeof(buffer), readFile) != NULL) {
    if (buffer[0] != '\n') {
        fprintf(blankFile, "%s", buffer);
    }
}

fclose(readFile);
fclose(blankFile);
}

```

### **Explanation:**

- The function opens the input file in read mode and the output file in write mode. Error handling ensures graceful termination if file opening fails.
- Using a loop, it reads each line from the input file using `fgets()` and stores it in the `buffer` array.
- If the first character of a line is not a newline character ('\n'), indicating a non-empty line, the line is written to the output file using `fprintf()`.
- Finally, both input and output files are closed to release system resources.

This algorithm effectively removes blank lines from the input file, producing a clean and compact output file without any unnecessary empty lines.

## **Section 2: Removing Comments**

The process of removing comments from a C source file involves identifying and eliminating both single-line and multi-line comments while preserving the actual code content. This section outlines the algorithm and logic behind the `removeComments` function.

### **Functionality:**

This function aims to eliminate comments from the content of the input file, ensuring that only code without comments is retained in the output file.

## Arguments:

- `inputFile`: Path to the input file containing text with comments.
- `outputFile`: Path to the output file where the content without comments will be written.

## Variables:

- `readFile`: File pointer for the input file.
- `noCommentFile`: File pointer for the output file without comments.
- `buffer`: Character array to store each line read from the input file.
- `blockComment`: Flag variable to track multi-line comment blocks.

## Code Snippet:

```
void removeComments(char *inputFile, char *outputFile) {
```

```
    FILE *readFile = fopen(inputFile, "r");
```

```
    if (readFile == NULL) {
```

```
        printf("Couldn't open %s\n", inputFile);
```

```
        exit(1);
```

```
    }
```

```
    FILE *noCommentFile = fopen(outputFile, "w");
```

```
    if (noCommentFile == NULL) {
```

```
        printf("Couldn't open %s\n", outputFile);
```

```
        exit(1);
```

```
    }
```

```
    char buffer[256];
```

```
    int blockComment = 0;
```

```
    while (fgets(buffer, sizeof(buffer), readFile) != NULL) {
```

```
        for (int i = 0; buffer[i] != '\0'; i++) {
```

```
            // Check for single line comments
```

```
            if (buffer[i] == '/' && buffer[i + 1] == '/') {
```

```

        break;
    }

    // Check for start of block comment
    if (!blockComment && buffer[i] == '/' && buffer[i + 1] == '*') {
        blockComment = 1;
        i++;
    }

    // Check for end of block comment
    else if (blockComment && buffer[i] == '*' && buffer[i + 1] == '/') {
        blockComment = 0;
        i+=2;
    }

    // Write to file if not inside a comment block
    else if (!blockComment) {
        fputc(buffer[i], noCommentFile);
    }
}

fclose(readFile);
fclose(noCommentFile);
}

```

### **Explanation:**

- The function opens the input file in read mode and the output file in write mode. Error handling ensures graceful termination if file opening fails.

- Using a loop, it reads each line from the input file using `fgets()` and stores it in the `buffer` array.
- Within a nested loop, it iterates through each character of the line to identify single-line and multi-line comments.
- Single-line comments are detected by encountering the character sequence `'/'` and are skipped.
- Multi-line comments are identified by the character sequence `'/*'` indicating the start of a block comment. The function sets the `blockComment` flag to 1 to indicate being inside a comment block.
- The function continues reading characters until it encounters the character sequence `'*/'` indicating the end of the comment block. The `blockComment` flag is then reset to 0, and the loop continues.
- Characters outside comment blocks are written to the output file using `fputc()`.
- Finally, both input and output files are closed to release system resources.

This algorithm effectively removes both single-line and multi-line comments from the input file, producing a clean output file with only the code content intact.

### **Section 3: Expanding Macros**

Macro expansion involves replacing occurrences of macro identifiers with their corresponding definitions throughout the code, effectively incorporating the macro values into the source code. This section outlines the algorithm and logic behind the `macroExpansion` function.

#### **Functionality:**

This function aims to expand macros defined in the input file, replacing macro identifiers with their corresponding values as specified in the source code.

#### **Arguments:**

- `inputFile`: Path to the input file containing C source code with macro definitions.
- `outputFile`: Path to the output file where the content with expanded macros will be written.

#### **Variables:**

- `readFile`: File pointer for the input file.
- `macrosFile`: File pointer for the output file with expanded macros.
- `buffer`: Character array to store each line read from the input file.
- `macros`: Array of structs to store macro name and value pairs.
- `numOfMacros`: Integer variable to track the number of macros defined in the input file.



**Code Snippet:**

```
void macroExpansion(char *inputFile, char *outputFile) {  
    char buffer[256];  
    struct Macro macros[100];  
    int numOfMacros = 0;  
  
    while (fgets(buffer, sizeof(buffer), readFile) != NULL) {  
        if (buffer[0] == '#' && buffer[1] == 'd') {  
            // Extract macro name and value  
            char macroName[256], macroValue[256];  
            stringScanf(buffer, "#define %s %[^\\n]", macroName, macroValue);  
  
            // Store macro in macros array  
            copyString(macros[numOfMacros].name, macroName);  
            copyString(macros[numOfMacros].value, macroValue);  
            numOfMacros++;  
        } else {  
            // Macro replacement in the line  
            int i = 0;  
            while (buffer[i] != '\\0') {  
                int foundMacro = -1;  
  
                // Check if current position matches any macro  
                for (int j = 0; j < numOfMacros; j++) {  
                    int k = 0;  
  
                    // Check for complete match of macro name
```

```

while (macros[j].name[k] != '\0' && buffer[i + k] == macros[j].name[k]) {
    k++;

// Macro found, replace it with its value
if (foundMacro != -1) {
    fputs(macros[foundMacro].value, macrosFile);

    i += strlen(macros[foundMacro].name); // Move index to after the macro name
}

```

### Explanation:

- The function opens the input file in read mode and the output file in write mode. Error handling ensures graceful termination if file opening fails.
- Using a loop, it reads each line from the input file using `fgets()` and stores it in the `buffer` array.
- If a line begins with the `#define` directive, indicating a macro definition, the function extracts the macro name and value and stores them in the `macros` array.
- For lines not containing macro definitions, the function iterates through each character to identify macro-occurrences. If a macro is found, it replaces it with its corresponding value from the `macros` array.
- Finally, both input and output files are closed to release system resources.

This algorithm effectively expands macros defined in the input file, incorporating their values into the source code, and producing an output file with fully expanded macros.

# OUTPUT

Usage: main <inputCodeFile> <noBlanksFile> <noCommentsFile> <macroExpandedFile>

## Sample 1:

Input:

- ./main sample.c noBlanks.c noComments.c macros.c:

Original sample1.c:

```
C sample.c > _
1  /****in1.c****
2  *
3  *
4  */
5  #include <stdio.h>
6  // defining macros
7
8
9  #define ON 1
10 #define OFF 0
11
12
13 void main()
14 {
15
16
17     /*declaring variables*/
18     int j = 2;
19     int motor, sensorValue = 0;
20     if (motor == ON) // what to do when motor is running
21     {
22         sensorValue++;
23     }
24     else if (motor == OFF) /* what to do when motor is not running */
25     {
26         sensorValue--;
27     }
28
29
30     return 0;
31 }
32
```

## Removed Blanks:

```
C noBlanks.c > _
1  /*****in1.c*****/
2  *
3  *
4  */
5  #include <stdio.h>
6  // defining macros
7  #define ON 1
8  #define OFF 0
9  void main()
10 {
11     /*declaring variables*/
12     int j = 2;
13     int motor, sensorValue = 0;
14     if (motor == ON) // what to do when motor is running
15     {
16         sensorValue++;
17     }
18     else if (motor == OFF) /* what to do when motor is not running */
19     {
20         sensorValue--;
21     }
22     return 0;
23 }
24
```

## Removed Comments:

```
C noComments.c > _
1  #include <stdio.h>
2  #define ON 1
3  #define OFF 0
4  void main()
5  {
6      int j = 2;
7      int motor, sensorValue = 0;
8      if (motor == ON) {
9          sensorValue++;
10     }
11     else if (motor == OFF) {
12         sensorValue--;
13     }
14     return 0;
15 }
16
```

## Expanded Macros:

```

C macro.c > ...
1  #include <stdio.h>
2  void main()
3  {
4      int j = 2;
5      int motor, sensorValue = 0;
6      if (motor == 1) {
7          sensorValue++;
8      }
9      else if (motor == 0) {
10         sensorValue--;
11     }
12     return 0;
13 }
14

```

## Sample 2:

Input:

- ./main sample2.c noBlanks2.c noComments2.c macros2.c:

Original sample2.c:

```

C sample2.c > ...
1  /*Program Name: in1.c
2  *Description:
3  *Input file for assignment 1
4  */
5  #include <stdio.h>
6  #include <string.h>
7  #include <stdlib.h>
8  //defining macros
9  #define MESSAGE1 "Hello class\n"
10 #define MESSAGE2 "Computer Science Department"
11
12
13 void main()
14 {
15
16     ////////////Print messages
17
18     printf(MESSAGE1);
19     printf(MESSAGE2);
20
21
22     return 0;
23 }

```

## Removed Blanks:

```
C noBlanks.c > _
1  /*Program Name: in1.c
2  *Description:
3  *Input file for assignment 1
4  */
5  #include <stdio.h>
6  #include <string.h>
7  #include <stdlib.h>
8  //defining macros
9  #define MESSAGE1 "Hello class\n"
10 #define MESSAGE2 "Computer Science Department"
11 void main()
12 {
13     //Print messages
14     printf(MESSAGE1);
15     printf(MESSAGE2);
16     return 0;
17 }
18
```

## Removed Comments:

```
C noComments.c > _
1  #include <stdio.h>
2  #include <string.h>
3  #include <stdlib.h>
4  #define MESSAGE1 "Hello class\n"
5  #define MESSAGE2 "Computer Science Department"
6  void main()
7  {
8      printf(MESSAGE1);
9      printf(MESSAGE2);
10     return 0;
11 }
12
```

## Expanded Macros:

```
C macro.c > ...  
1  #include <stdio.h>  
2  #include <string.h>  
3  #include <stdlib.h>  
4  void main()  
5  {  
6      printf("Hello class\n");  
7      printf("Computer Science Department");  
8      return 0;  
9  }  
10
```

Emulating how preprocessor works, the removal of blank lines, comments, and the expansion of macros exemplify the program's capability to process source code intelligently, producing refined and optimized output. Each section of the program's functionality, meticulously documented and explained, contributes to its overall robustness and efficiency.